Word Embeddings

We will discuss CBOW, Skip-Gram, Negative Sampling, and GloVe.

1 Continuous Bag of Words (CBOW)

Word2Vec is an algorithm for generating word embeddings, mapping words from a large corpus to vectors in a continuous vector space. Words that have similar meanings are placed close to one another in this vector space. Word2Vec has two main architectures:

- Continuous Bag of Words (CBOW)
- Skip-gram

1.1 Loss Function for CBOW

The CBOW predicts a target word w_t based on its surrounding context using:

$$P(w_t|C) = \frac{\exp(\mathbf{v}_{w_t}^T \cdot \mathbf{u}_C)}{\sum_{w \in V} \exp(\mathbf{v}_w^T \cdot \mathbf{u}_C)},$$

where $\mathbf{u}_C = \frac{1}{|C|} \sum_{w_c \in C} \mathbf{u}_{w_c}$. The loss function is:

$$\mathcal{L}_{\text{CBOW}} = -\sum_{t=1}^{T} \log P(w_t|C).$$

where:

- T is the number of target words.
- $p(w_t|\text{context})$ is the predicted probability of the target word w_t given the context words.

1.2 CBOW Network Architecture

The context of a word goes into the network and outputs the word (center)!

1.2.1 Step 1: Input Layer

Each **context word** w_c is represented as a **one-hot encoded vector** of size V (where V is the vocabulary size). If there are $2m$ context words, the input consists of:

$$x_{w_{t-m}}, x_{w_{t-m+1}}, ..., x_{w_{t+m-1}}, x_{w_{t+m}}$$

Each x_{w_c} is a V-dimensional **one-hot vector** where only the position corresponding to the word index is 1, and all other entries are 0.

1.2.2 Step 2: Hidden Layer (Word Embedding)

The one-hot vectors are multiplied by a **weight matrix** W, which is the **word embedding matrix**:

$$W \in \mathbb{R}^{V \times d}$$

where:

- V = Vocabulary size
- d = Embedding dimension

Since each input word is **one-hot**, selecting its embedding reduces to extracting the corresponding row from W.

$$h_{w_c} = W^T x_{w_c}$$

To obtain a single **hidden representation**, the CBOW model **averages the embeddings** of the context words:

$$h = \frac{1}{2m} \sum_{c=1}^{2m} W^T x_{w_c}$$

Thus, h is a d-dimensional vector representing the averaged context word embeddings.

1.2.3 Step 3: Output Layer (Softmax)

The output layer consists of another weight matrix W', which maps the hidden representation h back to a probability distribution over all words in the vocabulary:

$$u = W'h$$

where:

$$W' \in \mathbb{R}^{d \times V}$$

Finally, softmax is applied to obtain the probability distribution over all words in the vocabulary:

$$P(w_t|C_t) = \frac{\exp({W'}_{w_t}^T h)}{\sum_{v \in V} \exp({W'}_v^T h)}$$

where:

- $P(w_t|C_t)$ is the probability of predicting w_t given the context words.
- ${W'}_{w_t}^T h$ gives the score for word w_t before applying softmax.

1.2.4 Loss Function

CBOW is trained using **negative log-likelihood loss** to maximize the probability of the correct target word:

$$L = -\log P(w_t|C_t)$$

Expanding using softmax:

$$L = -\left({W'}_{w_t}^T h - \log \sum_{v \in V} \exp({W'}_v^T h) \right)$$

1.2.5 Summary of CBOW Network Architecture

Layer	Operation
Input Layer	One-hot encoded context words (size V)
Hidden Layer	Word embeddings (size d), computed as the **average** of context word embeddings
Output Layer	Softmax over vocabulary to predict target word

Table 1: CBOW Network Architecture

2 Skip-gram

2.1 Loss Function for Skip-gram

The loss function for Skip-gram is similar to CBOW, but here we aim to predict multiple context words. It is given by:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T}\sum_{c\in \text{context}(w_t)} \log p(c|w_t)$$

Where:

- T is the number of target words.
- context(w_t) represents the set of context words for the target word w_t.
- $p(c|w_t)$ is the predicted probability of the context word c given the target word w_t.

The objective is to maximize the likelihood of predicting the correct context words for each target word.

Skip-Gram:

Predicts context words based on a target word w_t.

$$P(w_c|w_t) = \frac{\exp(\mathbf{v}_{w_c}^T \cdot \mathbf{u}_{w_t})}{\sum_{w\in V}\exp(\mathbf{v}_w^T \cdot \mathbf{u}_{w_t})}.$$

The loss function is:

$$\mathcal{L}_{\text{Skip-Gram}} = -\sum_{t=1}^{T}\sum_{c\in C}\log P(w_c|w_t).$$

2.2 Skip-Gram Network Architecture

2.2.1 Overview of Skip-Gram

The **Skip-Gram** model, part of the **Word2Vec** framework, is a neural network that learns word embeddings by predicting context words given a center word. Unlike **CBOW**, which predicts the target word given its surrounding context, **Skip-Gram** uses a single word to predict multiple context words.

Given a **center word** w_t, Skip-Gram learns to predict its **context words** w_{t+j}, where j is within a defined window size.

For example, in the sentence:

"The cat sat on the mat."

If the **window size** is 2, the model could be trained on:

- **Center word**: "cat"
- **Context words**: ("The", "sat", "on", "the")

The goal is to **maximize the probability of correctly predicting context words given the center word**.

2.2.2 Network Architecture of Skip-Gram

2.2.2.1 Step 1: Input Layer

Each **center word** w_t is represented as a **one-hot encoded vector** of size V (where V is the vocabulary size). The input is:

$$x_{w_t} = [0, \ldots, 0, 1, 0, \ldots, 0]^T$$

where only the position corresponding to w_t is 1.

2.2.2.2 Step 2: Hidden Layer (Word Embedding)

The one-hot vector is multiplied by a **weight matrix** W, which is the **word embedding matrix**:

$$W \in \mathbb{R}^{V \times d}$$

where:

- V is the vocabulary size.
- d is the embedding dimension.

Since the input is **one-hot**, the hidden representation **selects the row corresponding to** w_t **from** W:

$$h = W^T x_{w_t}$$

Thus, h is a d-dimensional vector representing the embedding of the center word w_t.

2.2.2.3 Step 3: Output Layer (Softmax)

The hidden representation h is passed through another weight matrix W', which maps it back to a probability distribution over all words in the vocabulary:

$$u = W'h$$

where:

$$W' \in \mathbb{R}^{d \times V}$$

Softmax is then applied to compute the probability distribution over all words:

$$P(w_o|w_t) = \frac{\exp({W'}_{w_o}^T h)}{\sum_{v \in V} \exp({W'}_v^T h)}$$

where:

- $P(w_o|w_t)$ is the probability of predicting context word w_o given center word w_t.
- ${W'}_{w_o}^T h$ gives the score for word w_o before applying softmax.

Since multiple context words are predicted for each center word, this step is repeated for every context word in the window.

2.2.3 Loss Function

The Skip-Gram model is trained using **negative log-likelihood loss**, which maximizes the probability of predicting correct context words given the center word:

$$L = -\sum_{w_o \in C_t} \log P(w_o|w_t)$$

Expanding using softmax:

$$L = -\sum_{w_o \in C_t} \left({W'}_{w_o}^T W_{w_t} - \log \sum_{v \in V} \exp({W'}_v^T W_{w_t}) \right)$$

2.2.4 Summary of Skip-Gram Network Architecture

Layer	Operation
Input Layer	One-hot encoded center word (size V)
Hidden Layer	Word embedding (size d), extracted from the embedding matrix
Output Layer	Softmax over vocabulary to predict context words

Table 2: Skip-Gram Network Architecture

2.2.5 Key Differences Between Skip-Gram and CBOW

Feature	Skip-Gram	CBOW
Objective	Predict context words given center word	Predict center word given context words
Computation	Slower (multiple predictions per word)	Faster (single prediction per window)
Works well with	Small datasets	Large datasets

Table 3: Comparison of Skip-Gram and CBOW

3 Efficient Computation: Negative Sampling and Hierarchical Softmax

For large vocabularies, computing the full softmax over the entire vocabulary can be expensive. To address this, Word2Vec employs two efficient techniques:

3.1 Negative Sampling

Instead of updating the softmax over all words in the vocabulary, the model uses negative sampling, where only a small set of negative samples (random words) are drawn for each positive word pair (target word, context word).

The loss function for negative sampling in Word2Vec (specifically, for Skip-gram, though the principle can apply to CBOW) can be described as follows:

- **Negative Sampling**: Simplifies the softmax computation by sampling a small number of "negative" words not in the context.

$$\mathcal{L}_{\text{Negative Sampling}} = -\sum_{t}\sum_{j=-C}^{C,j\neq 0}\left[\log\sigma(\mathbf{v}_{w_t}^T\mathbf{u}_{w_{t+j}}) + \sum_{k=1}^{K}\log\sigma(-\mathbf{v}_{w_k}^T\mathbf{u}_{w_{t+j}})\right].$$

  Where:

  - σ is the sigmoid function, defined as $\sigma(x) = \frac{1}{1+e^{-x}}$.
  - $\mathbf{v}_{w_t}$ is the **target word vector** (from W).
  - $\mathbf{u}_{w_{t+j}}$ is the **context word vector** (from W').
  - Here, w_{t+j} is a context word, and j iterates over the context window from $-C$ to C excluding 0, where C is the context window size.
  - $\mathbf{v}_{w_k}$ are the vectors of the **negative samples** (also from W). These are words drawn from the vocabulary excluding the true context words, with K being the number of negative samples per positive example.

3.2 Hierarchical Softmax

Hierarchical Softmax is an optimization technique used in large vocabulary neural network models, particularly in word embedding algorithms like Word2Vec (both Continuous Bag of Words (CBOW) and Skip-gram). It addresses the computational inefficiency of the traditional softmax function by employing a binary tree structure to reduce the number of computations required for each training example or prediction.

- **Computational Efficiency**: With traditional softmax, computing the probability for a word involves normalizing over all words in the vocabulary, leading to $O(V)$ complexity where V is the vocabulary size. Hierarchical softmax reduces this to $O(\log V)$.
- **Memory Efficiency**: It can also lead to a reduction in the number of parameters if the tree is constructed wisely, like using Huffman coding to minimize the depth for frequent words.

1. **Tree Structure**:
   - Each word in the vocabulary is placed at a leaf node of a binary tree.
   - Internal nodes represent binary decisions (left or right) that guide us to the correct word.
2. **Tree Construction**:

- Ideally, the tree should be balanced or constructed based on word frequencies (like Huffman coding) to minimize the average path length to frequent words.

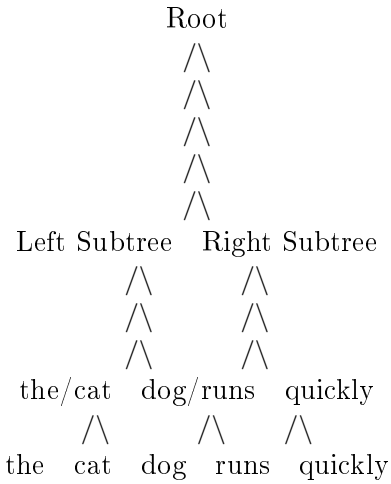
3. **Prediction and Training**:

   - **Prediction**: To predict a word or compute its probability, you navigate from the root to the leaf representing the word:
     - At each internal node, a decision (left or right) is made based on the dot product of the context vector **h** and the vector associated with the node, followed by a sigmoid function to determine the probability of going left.
   - **Training**: The model learns weights for each node so that the path to the correct word becomes more probable given the context.

Vocabulary: Let's use a small vocabulary for illustration:

- "the"
- "cat"
- "dog"
- "runs"
- "quickly"

Constructing the Tree:
For simplicity, we'll use a balanced binary tree:



Explanation:

- **Root Node**: The first decision point. Here, we decide whether to look for "the", "cat", "dog", "runs" (left subtree) or "quickly" (right subtree).
- **First Level Nodes**:
  - **Left Node**: Further splits into "the"/"cat" and "dog"/"runs".
  - **Right Node**: Directly leads to "quickly" since it's the only word in this branch.
- **Second Level Nodes**:

- **Left-Left Node**: Splits into "the" and "cat".
- **Left-Right Node**: Splits into "dog" and "runs".

Loss Function:
The loss function for a word like "cat" given some context would involve:

- **Context Vector h**:
  - For CBOW, it's the average of context word embeddings.
  - For Skip-gram, it's the embedding of the target word itself.
- **Path to "cat"**:
  - Root to Left Subtree (left decision)
  - Left Subtree to "the/cat" (left decision)
  - "the/cat" to "cat" (right decision)

The loss is computed as:

$$\mathcal{L}_{\text{"cat"}} = -\left(\log(\sigma(v_{\text{Root}}^T \cdot \mathbf{h})) + \log(\sigma(v_{\text{Left Subtree}}^T \cdot \mathbf{h})) + \log(\sigma(-v_{\text{The/Cat}}^T \cdot \mathbf{h}))\right)$$

Where σ is the sigmoid function, ensuring the probability for each correct decision is maximized.
Training Details:

- Each node in the tree has an associated vector.
- During training, these vectors are adjusted to make the correct path to each word more likely given the context.
- The model essentially learns to make a series of binary decisions that lead to the correct word.

Advantages and Considerations:

- **Scalability**: Works well with large vocabularies due to reduced computational complexity.
- **Balancing**: The efficiency can be greatly affected by how the tree is balanced or constructed.
- **Interpretability**: While less intuitive than traditional softmax, it provides a structured method for word prediction.

Hierarchical softmax is a powerful method for scaling word embedding models to large vocabularies by transforming the problem of computing probabilities over all words into a series of binary decisions. It's crucial in applications where computational resources are a constraint, allowing for more efficient training and inference processes.

4 What is GloVe?

GloVe (Global Vectors for Word Representation) is a popular method for generating word embeddings, introduced by Pennington, Socher, and Manning in 2014. It creates dense, low-dimensional vector representations of words that capture semantic and syntactic relationships. Unlike Word2Vec, which is based on a predictive approach, GloVe uses a **count-based approach** to leverage the global statistical properties of a corpus.

4.1 Key Features of GloVe

- **Global Context**: Captures the global co-occurrence statistics of words in a corpus using a co-occurrence matrix.
- **Semantic Relationships**: Like Word2Vec, GloVe embeddings capture relationships between words. For example:
$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}.$$
- **Explicit Co-occurrence**: Directly models the co-occurrence matrix, making it more efficient in leveraging corpus-wide statistics.

How GloVe Works

1. Co-occurrence Matrix

GloVe constructs a word co-occurrence matrix X, where:

- X_{ij} represents how often word i appears in the context of word j.
- The context can be defined by a window size or other criteria.

2. Objective Function

GloVe defines a weighted least-squares regression problem to find word vectors $\mathbf{v}_i$ and context word vectors $\mathbf{u}_j$ such that:
$$f(X_{ij})(\mathbf{v}_i^T \mathbf{u}_j + b_i + b_j - \log X_{ij})^2,$$
where:

- $\mathbf{v}_i$: Word vector for word i.
- $\mathbf{u}_j$: Context vector for word j.
- b_i, b_j: Bias terms for the word and context vectors.
- $f(X_{ij})$: A weighting function to reduce the impact of very frequent co-occurrences:
$$f(X_{ij}) = \begin{cases} \left(\frac{X_{ij}}{x_{\max}}\right)^{\alpha} & \text{if } X_{ij} \leq x_{\max}, \\ 1 & \text{otherwise.} \end{cases}$$
- $x_{\max}$: A threshold to limit the influence of large co-occurrence counts.
- α: A hyperparameter, typically set to 0.75.

3. Optimization

GloVe minimizes the loss function using stochastic gradient descent (SGD) or similar optimization techniques. The final embedding for a word is typically computed by summing its word vector and context vector:
$$\mathbf{w}_i = \mathbf{v}_i + \mathbf{u}_i.$$

Strengths of GloVe

- **Global Context**: Leverages co-occurrence statistics to capture long-range dependencies.
- **Semantic Relationships**: Effectively encodes relationships between words, making it useful for analogy tasks.
- **Efficiency**: Precomputes the co-occurrence matrix, making training faster than iterative prediction-based methods like Word2Vec.
- **Pretrained Embeddings**: Offers high-quality pretrained embeddings for large corpora like Wikipedia and Common Crawl.

Limitations of GloVe

- **Context Independence**: Produces static embeddings, meaning a word has the same vector regardless of its context.
- **Memory Intensive**: Constructing and storing the co-occurrence matrix can be resource-intensive for large vocabularies.
- **Sensitivity to Preprocessing**: The quality of embeddings depends heavily on the corpus preprocessing (e.g., tokenization, stop-word removal).

Comparison with Word2Vec

Feature	Word2Vec	GloVe
Approach	Predictive (context $\rightarrow$ target)	Count-based (word co-occurrence)
Training Data	Local context	Global co-occurrence statistics
Optimization	Maximizes predictive probabilities	Minimizes weighted least-squares loss
Embedding Output	Static word embeddings	Static word embeddings
Training Efficiency	Iterative, slower	Faster for large corpora

Table 4: Comparison between Word2Vec and GloVe.

GloVe is a powerful method for generating word embeddings by leveraging global co-occurrence statistics. While it has limitations, such as context independence, it remains a foundational technique in NLP. Its pretrained embeddings are widely used due to their quality and availability.

Limitations of Word2Vec

- **Context Independence**: A word has the same embedding regardless of its surrounding context (e.g., "bank" as a financial institution vs. riverbank).
- **OOV Problem**: Cannot generate embeddings for words not seen during training.
- **Shallow Model**: Lacks the capacity to model deep, nuanced relationships compared to newer models like BERT or GPT.

GloVe does not resolve the fundamental limitations of Word2Vec, such as context independence , the OOV problem , or the shallow model architecture . However, it improves upon Word2Vec in other areas, such as leveraging global context , being more computationally efficient for large corpora, and addressing the imbalance caused by Zipf's Law through its weighting function.

For tasks requiring contextual embeddings or handling OOV words , neither Word2Vec nor GloVe is sufficient. Instead, modern transformer-based models like BERT , GPT , or ELMo are better suited, as they generate dynamic, context-aware embeddings and can handle unseen words using subword tokenization.

Thus, while GloVe offers some advantages over Word2Vec, it does not fully address its core limitations.